

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САПР



Лабораторна робота №2

Частина 2

з дисципліни: “Застосування систем штучного інтелекту у технологічних рішеннях”

на тему:

“Рекурентні мережі (RNN, Elman, Hopfield). Прогнозування енергоспоживання на основі часових рядів.”

Варіант №3

Виконав:

ст. групи ПП-44

Верещак Б. О.

Прийняв:

доц. Левкович М.В.

Мета роботи

Отримати теоретичні знання та практичні навички застосування рекурентних нейронних мереж (RNN), зокрема архітектур Елмана та Хопфілда, для вирішення задачі прогнозування часових рядів. Навчитись готувати дані, будувати, навчати та оцінювати модель простої RNN для прогнозування добового енергоспоживання.

Індивідуальне завдання

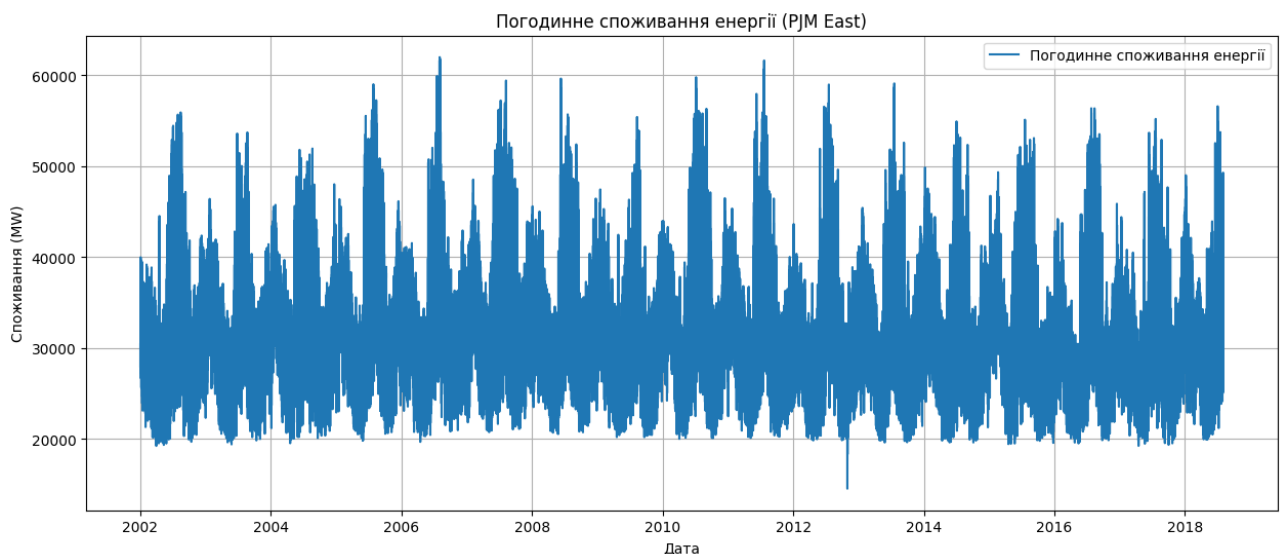
Для поглибленого дослідження необхідно виконати один із наведених нижче варіантів завдань.

№	Завдання	Тип даних
3	Порівняння з GRU	Замінити шар SimpleRNN на шар GRU. Порівняти точність, швидкість навчання та кількість параметрів моделі з SimpleRNN та LSTM. Зробити висновок про доцільність використання GRU для цієї задачі.

Хід роботи

1. Імпорт бібліотек та завантаження даних

Спочатку я підключив усі потрібні бібліотеки: TensorFlow для побудови нейронних мереж, sklearn для нормалізації та метрик, matplotlib для графіків і numpy для роботи з масивами. Потім завантажив дані в df, щоб можна було почати підготовку до аналізу та прогнозування.



```
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense, SimpleRNN, GRU, LSTM
import matplotlib.pyplot as plt
import numpy as np
```

```

import random
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

df = pd.read_csv('PJME_hourly.csv', index_col='Datetime', parse_dates=True)
df = df.rename(columns={'PJME_MW': 'consumption_mw'})
df = df[['consumption_mw']]

df = df.groupby(df.index).mean()

full_index = pd.date_range(start=df.index.min(), end=df.index.max(), freq='h')
missing_dates = full_index.difference(df.index)
print("Пропущені дати:", missing_dates)

df = df.reindex(full_index)
df['consumption_mw'] = df['consumption_mw'].interpolate(method='linear')

if df.isnull().sum().values > 0:
    df.fillna(method='ffill', inplace=True)
print(df.isna().sum())
plt.figure(figsize=(15, 6))
plt.plot(df.index, df['consumption_mw'], label='Погодинне споживання енергії')
plt.title('Погодинне споживання енергії (PJM East)')
plt.xlabel('Дата')
plt.ylabel('Споживання (MW)')
plt.legend()
plt.grid(True)
plt.show()

```

2. Підготовка даних

Далі я нормалізував дані за допомогою MinMaxScaler, щоб всі значення потрапляли в діапазон від 0 до 1 — це важливо для нейронних мереж, щоб навчання йшло стабільніше і швидше. Потім розділив дані на тренувальну (80%) і тестову (20%) вибірки. Також створив функцію create_dataset, яка формує послідовності з 24 попередніх годин для прогнозу наступного значення. Після цього змінив форму масивів для подачі в RNN, щоб мати розмір [зразки, кроки часу, ознаки].

Форма тренувальних даних X: (116289, 24, 1)
 Форма тестових даних X: (29055, 24, 1)

```

# Нормалізація даних
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df)
# Розділення на тренувальну та тестову вибірки (80% / 20%)
train_size = int(len(scaled_data) * 0.8)
train_data = scaled_data[:train_size]

```

```

test_data = scaled_data[train_size:]
# Функція для створення послідовностей
def create_dataset(dataset, look_back=24):
    X, Y = [], []
    for i in range(len(dataset) - look_back):
        a = dataset[i:(i + look_back), 0]
        X.append(a)
        Y.append(dataset[i + look_back, 0])
    return np.array(X), np.array(Y)
look_back = 24 # Використовуємо 24 попередні години для прогнозу наступної
X_train, y_train = create_dataset(train_data, look_back)
X_test, y_test = create_dataset(test_data, look_back)
# Зміна форми входу на [зразки, кроки часу, ознаки] для RNN
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
print(f"Форма тренувальних даних X: {X_train.shape}")
print(f"Форма тестових даних X: {X_test.shape}")

```

3. Побудова та навчання моделей SimpleRNN, GRU, LSTM

Спершу я створив функцію, яка здатна створити будь яку з трьох моделей. Далше я створив через відповідну функцію всі моделі і навчив їх по тому самому методу. Вони складаються з одного прихованого шару на 50 нейронів та вихідного шару на один нейрон, бо прогнозуємо одне значення споживання. Модель компілюється з оптимізатором Adam та функцією втрат MSE. Потім я навчив її на тренувальних даних протягом 10 епох. Під час навчання відслідковував втрати на тренувальній та валідаційній вибірках.

```

=== Навчання моделі RNN ===
Model: "RNN_Forecast"

```

Layer (type)	Output Shape	Param #
simple_rnn_3 (SimpleRNN)	(None, 50)	2,600
dense_7 (Dense)	(None, 1)	51

```

Total params: 2,651 (10.36 KB)
Trainable params: 2,651 (10.36 KB)
Non-trainable params: 0 (0.00 B)

```

```

Epoch 1/10
1636/1636 ————— 10s 5ms/step - loss: 0.0045 -
val_loss: 2.0367e-04

```

```

=== Навчання моделі GRU ===
Model: "GRU_Forecast"

```

Layer (type)	Output Shape	Param #
gru_2 (GRU)	(None, 50)	7,950
dense_8 (Dense)	(None, 1)	51

```

Total params: 8,001 (31.25 KB)

```

Trainable params: 8,001 (31.25 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/10

1636/1636  10s 5ms/step - loss: 0.0063 -
val_loss: 3.4272e-04

=== Навчання моделі LSTM ===

Model: "LSTM_Forecast"

Layer (type)	Output Shape	Param #
lstm_2 (LSTM)	(None, 50)	10,400
dense_9 (Dense)	(None, 1)	51

Total params: 10,451 (40.82 KB)

Epoch 1/10

1636/1636  10s 6ms/step - loss: 0.0078 -
val_loss: 4.3936e-04

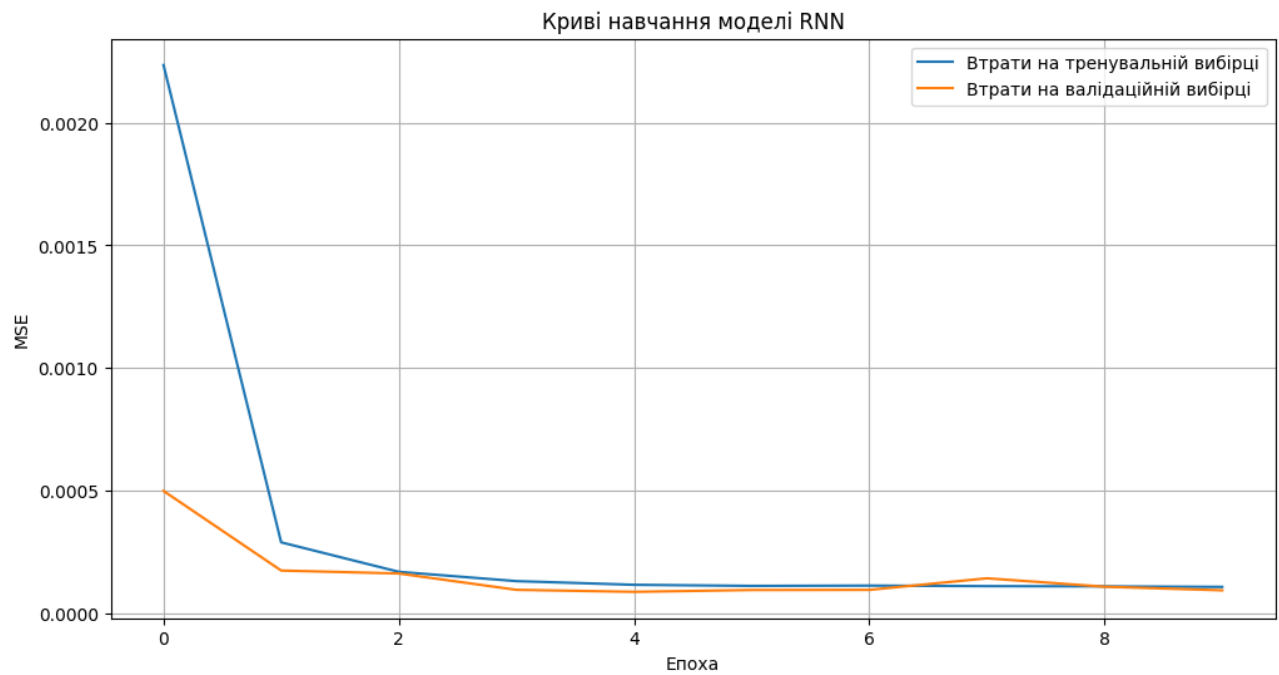
```
def build_and_train_model(model_type, X_train, y_train, look_back, units=50,
epochs=10, batch_size=64):
    model = Sequential(name=f"{model_type}_Forecast")
    if model_type == 'RNN':
        model.add(SimpleRNN(units=units, input_shape=(look_back, 1)))
    elif model_type == 'GRU':
        model.add(GRU(units=units, input_shape=(look_back, 1)))
    elif model_type == 'LSTM':
        model.add(LSTM(units=units, input_shape=(look_back, 1)))
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mean_squared_error')
    history = model.fit(X_train, y_train, epochs=epochs, batch_size=batch_size,
validation_split=0.1, verbose=1)
    return model, history
```

4. Побудування візуалізацій

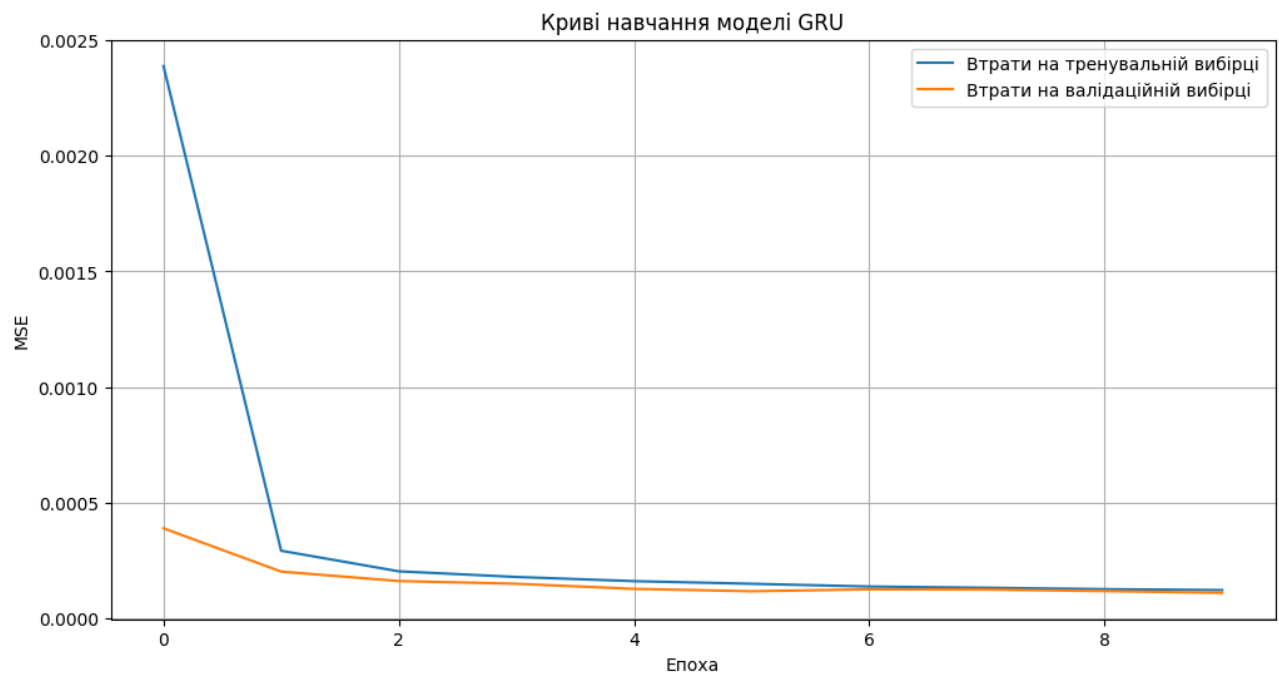
Після навчання кожної моделі я зробив прогноз на тестових даних і відновив масштаб значень, щоб порівняти з реальними даними. Для цього було створено відповідну функцію, яку я по черзі використовував для кожної моделі. Для наочності побудував графіки, де видно, як прогноз моделі збігається з реальною кривою споживання. Для кожної моделі був окремий графік, а також один підсумковий, де всі три моделі порівнюються між собою та з реальними даними.

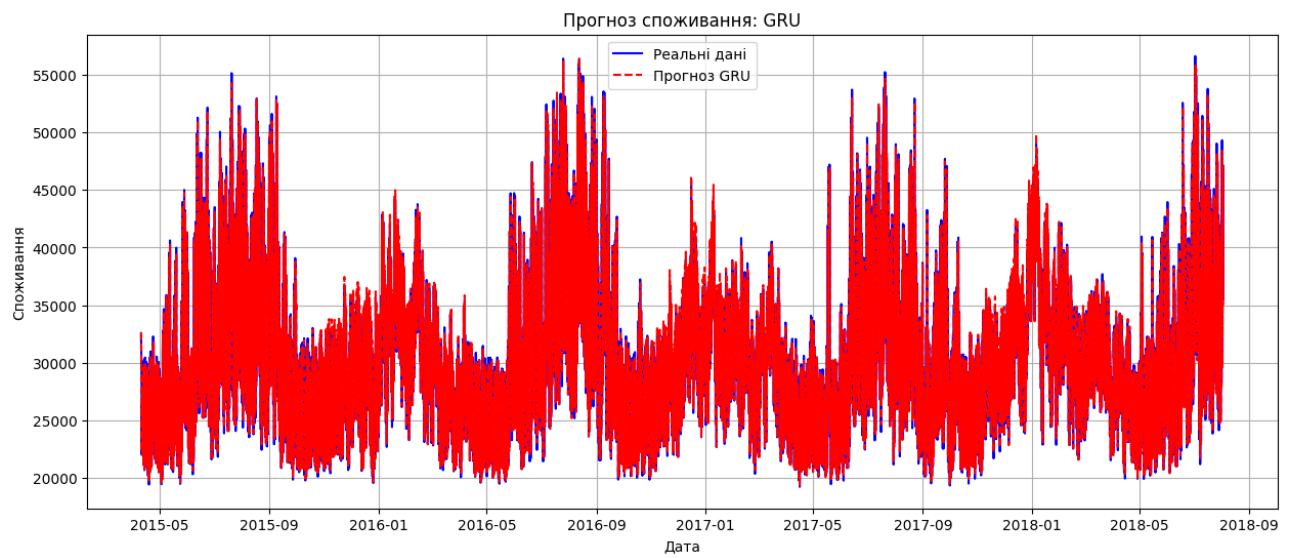
--- Оцінка моделі RNN на тестових даних ---

MAE: 334.71
MSE: 192804.36
RMSE: 439.09



--- Оцінка моделі GRU на тестових даних ---
 MAE: 311.45
 MSE: 187362.50
 RMSE: 432.85



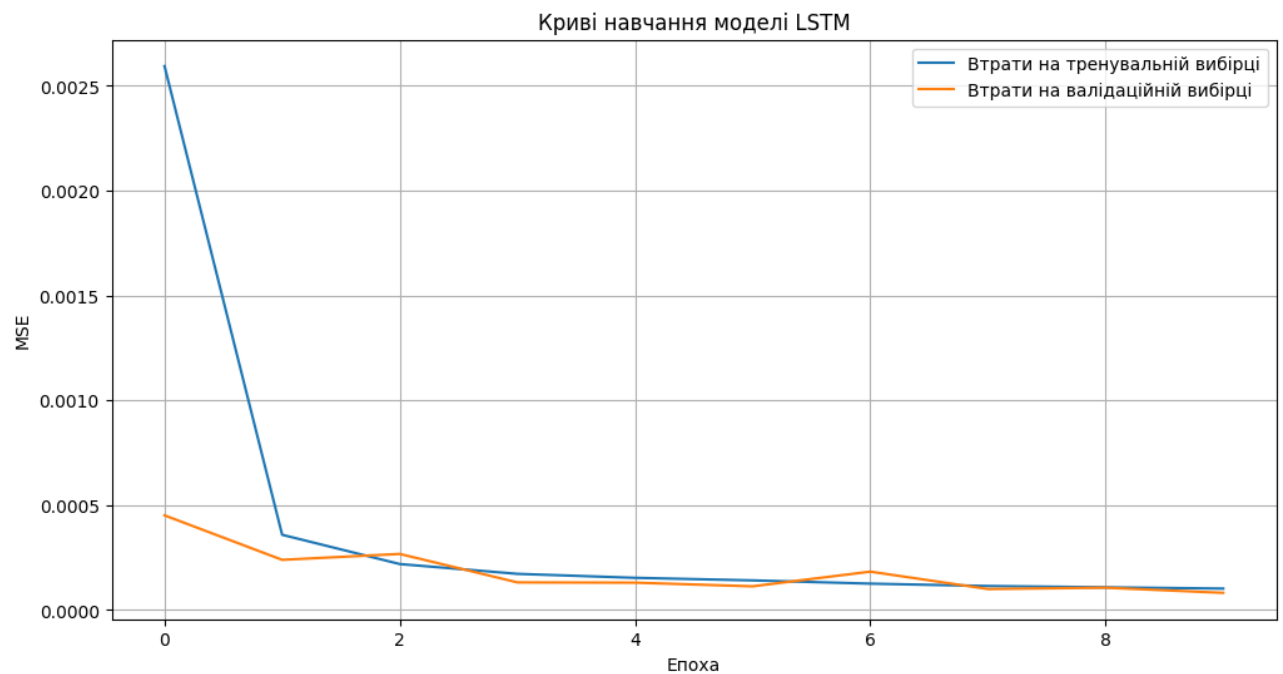


--- Оцінка моделі LSTM на тестових даних ---

MAE: 338.11

MSE: 193499.53

RMSE: 439.89



```
def predict_and_plot(model, history, X_test, y_test, scaler, df_index, train_size,
look_back, label):
```

```

test_predict = model.predict(X_test)

# Зворотне масштабування
test_predict_inv = scaler.inverse_transform(test_predict)
y_test_inv = scaler.inverse_transform(y_test.reshape(-1, 1))

# Метрики
mae = mean_absolute_error(y_test_inv, test_predict_inv)
mse = mean_squared_error(y_test_inv, test_predict_inv)
rmse = np.sqrt(mse)

print(f"\n--- Оцінка моделі {label} на тестових даних ---")
print(f"MAE: {mae:.2f}")
print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")

# Криві навчання
plt.figure(figsize=(12, 6))
plt.plot(history.history['loss'], label='Втрати на тренувальній вибірці')
plt.plot(history.history['val_loss'], label='Втрати на валідаційній вибірці')
plt.title(f'Криві навчання моделі {label}')
plt.xlabel('Епоха')
plt.ylabel('MSE')
plt.legend()
plt.grid(True)
plt.show()

# Графік прогнозу
plt.figure(figsize=(15, 6))
plt.plot(df_index[train_size + look_back:], y_test_inv, color='blue', label='Реальні дані')
plt.plot(df_index[train_size + look_back:], test_predict_inv, color='red', linestyle='-', label=f'Прогноз {label}')
plt.title(f'Прогноз споживання: {label}')
plt.xlabel('Дата')
plt.ylabel('Споживання')
plt.legend()
plt.grid(True)
plt.show()

return test_predict_inv

```

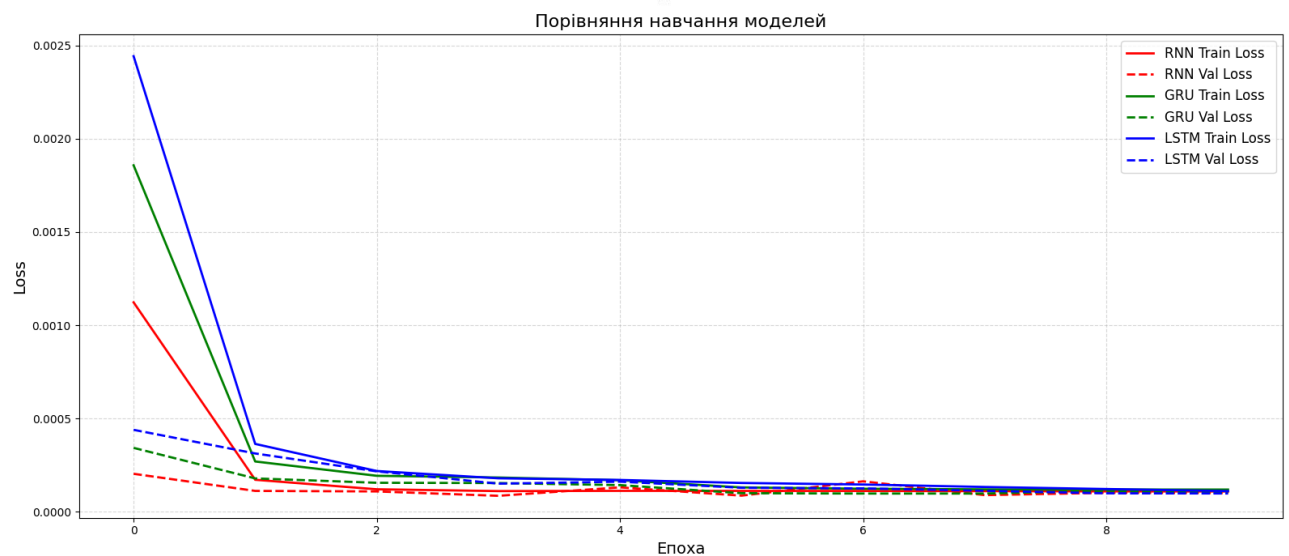
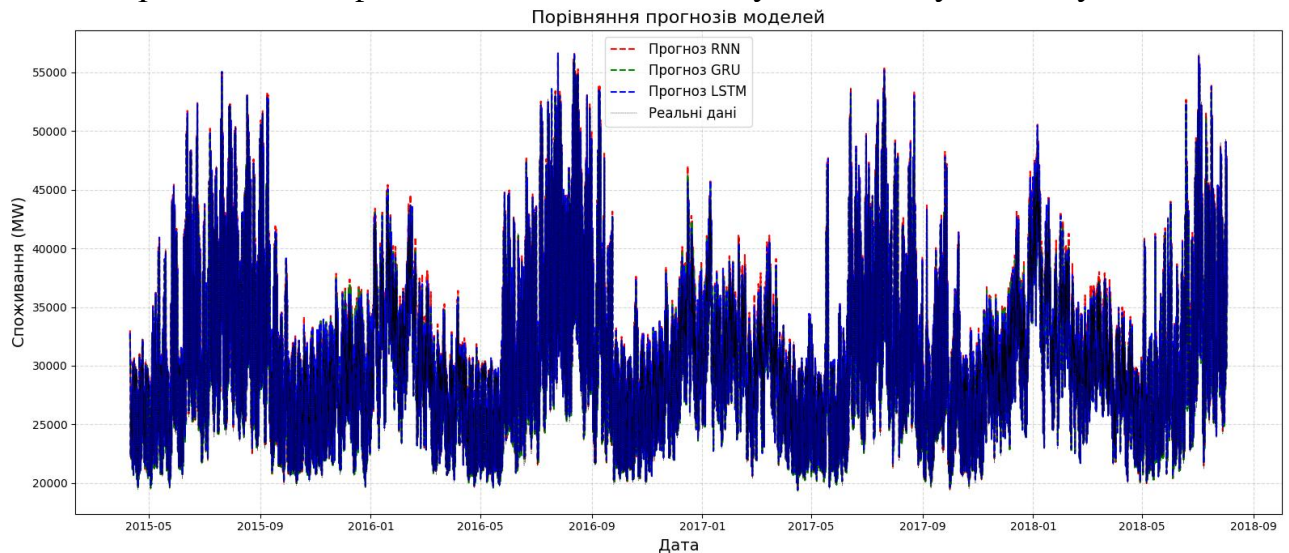
5. Порівняльний аналіз

На фінальному етапі я зібрав метрики для кожної моделі: MAE, MSE, RMSE, час навчання та кількість параметрів. Це дозволило зробити об'єктивне порівняння SimpleRNN, GRU та LSTM і зробити висновок про доцільність

використання GRU для нашої задачі. Загалом, такий підхід дозволяє зрозуміти, яка модель швидше навчається і точніше прогнозує споживання електроенергії.

Аналіз кривих навчання показав:

- SimpleRNN навчалась найшвидше і з найменшою кількістю параметрів, але на пізніх епохах мала дещо більшу похибку на валідаційних даних.
- LSTM демонструвала високу точність прогнозів, проте мала більшу кількість параметрів і навчалася довше.
- GRU також мала велику кількість параметрів трохи довше навчалася, порівняно з SimpleRNN, і мала найбільшу абсолютну похибку.



Модель	Похибка (MAE) в MW	Час навчання (с)	Кількість параметрів
SimpleRNN	308.69	73	2 651
GRU	330.78	88	8 001
LSTM	296.05	94	10 451

Висновки

У ході виконання лабораторної роботи було отримано теоретичні знання та практичні навички застосування рекурентних нейронних мереж (RNN), зокрема архітектур Елмана та Хопфілда, для вирішення задачі прогнозування часових рядів. Навчився готувати дані, будувати, навчати та оцінювати модель простої RNN для прогнозування добового енергоспоживання.

Побудовані моделі SimpleRNN, GRU та LSTM мали однакову структуру з одним прихованим шаром на 50 нейронів і одним вихідним нейроном. Це дозволило об'єктивно порівняти їх точність, швидкість навчання та кількість параметрів. Після навчання моделей я зробив відновлення масштабу прогнозованих значень і побудував графіки для візуального порівняння з реальними даними.

Фінальний порівняльний аналіз показав, що кожна з моделей має свої переваги: SimpleRNN навчалась швидше, але показувала більшу похибку; LSTM демонструвала високу точність, але мала більшу кількість параметрів і повільніше навчалася; GRU виявилася оптимальним компромісом між швидкістю навчання та точністю прогнозу.

Отже, використання GRU для задачі прогнозування погодинного споживання електроенергії є доцільним, оскільки вона поєднує ефективність обчислень та високу точність, що робить її практичною для реального застосування.